

Signal Object Detection

Faculty of Mathematics and Computer Science

Artificial Intelligence

Group: 241

Rachieriu Gheorghe Gabriel

June 8, 2026

Contents

1	First Attempt: K-NN	2
1.1	Why K-NN	2
1.2	Python Code	2
1.3	Confusion Matrix: Results	3
2	Second Attempt: SVM	4
2.1	About the approach	4
2.2	Confusion Matrix for SVM with no feature extraction: Results	4
2.3	Confusion Matrix for SVM with HOG: Results	5
2.4	Python Code	5
3	Last Attempt: CNN	6
3.1	Why I have choose this	6
3.2	Data Preprocessing: CLAHE	6
3.3	Data Augmentation	7
3.4	Neural Network Architecture	7
3.4.1	Visible Input First Layer	7
3.4.2	Hidden Convolution Blocks	7
3.4.3	Hidden Global Average Pooling	7
3.4.4	Visible Output Classification Layer	7
3.5	Dropout	7
3.6	Training and Optimization	8
3.7	Label Smoothing Loss	8
3.8	Learning Rate Scheduler	8
3.9	K-Fold Cross Validation	8
3.10	Early Stopping	8
3.11	Confusion Matrix: Results	9
3.12	Trying to optimize this model	11
3.12.1	Trying Optimizers	11
3.12.2	Trying Learning Rates	11
3.12.3	Trying different Schedulers	11
3.12.4	Trying Weight Decays	11
3.13	Gradient Clipping	11
3.14	Test Time Augmentation	12
3.15	Another Convolution Layer	12
3.16	Different Kernel Sizes	12
3.17	Python Code	12
4	Final Results	17

1 First Attempt: K-NN

1.1 Why K-NN

At first, I understood the classification completely wrong, thinking the model should detect **how many** objects are in the data (like vertical/horizontal/diagonal lines or intersections) and my first approach was already implemented in previous laboratory work K-NN clustering. My first submission score for this algorithm was under the baseline even after trying both Manhattan and Euclidean Distance but the scores were low for both.

1.2 Python Code

```
import numpy as np
import pandas as pd
import cv2
import os

class knn:
    def __init__(self, train_images, train_labels):
        self.train_images = train_images
        self.train_labels = train_labels

    def clasificare(self, test_image, k, metrica):
        if metrica == 'l2': #calculul distantei euclidiene
            distanta = np.sqrt(np.sum((self.train_images - test_image) ** 2, axis=1))
        elif metrica == 'l1': #calculul distantei manhattan
            distanta = np.sum(np.abs(self.train_images - test_image), axis=1)

        nearest_indices = np.argsort(distanta) #sortate crescator
        nearest_indices = nearest_indices[:k] # primii k vecini indici

        nearest_labels = self.train_labels[nearest_indices]
        voturi = np.bincount(nearest_labels)
        return np.argmax(voturi) # clasa cu cele mai multe voturi

def img(csv_path, imgs_path, label=True):
    df = pd.read_csv(csv_path)
    imagini = []
    labels = []

    for index, row in df.iterrows():
        img_path = os.path.join(imgs_path, row['id'])

        imagine = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE) #incarcam imaginea pe un singur canal
        img_flatten = imagine.flatten() # 2d -> 1d vector # adica doar greyscale
        imagini.append(img_flatten)

        if label:
            labels.append(row['label'])
    if label:
        return np.array(imagini), np.array(labels)
    else:
        return np.array(imagini), df

train_csv_path = ('/Users/gabrielrachieru/PycharmProjects/'
                  'IA-FMI-2026/SignalObjectDetection-Kaggle/signal-object-detection/train.csv')
train_imgs_path = ('/Users/gabrielrachieru/PycharmProjects'
                   '/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-object-detection/train')
```

```

test_csv_path = ('/Users/gabrielrachieru/PycharmProjects'
                 '/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-object-detection/test.csv')
test_imgs_path = ('/Users/gabrielrachieru/PycharmProjects'
                  '/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-object-detection/test')

train_images, train_labels = img(train_csv_path, train_imgs_path, label=True)
test_images, test_df = img(test_csv_path, test_imgs_path, label=False)

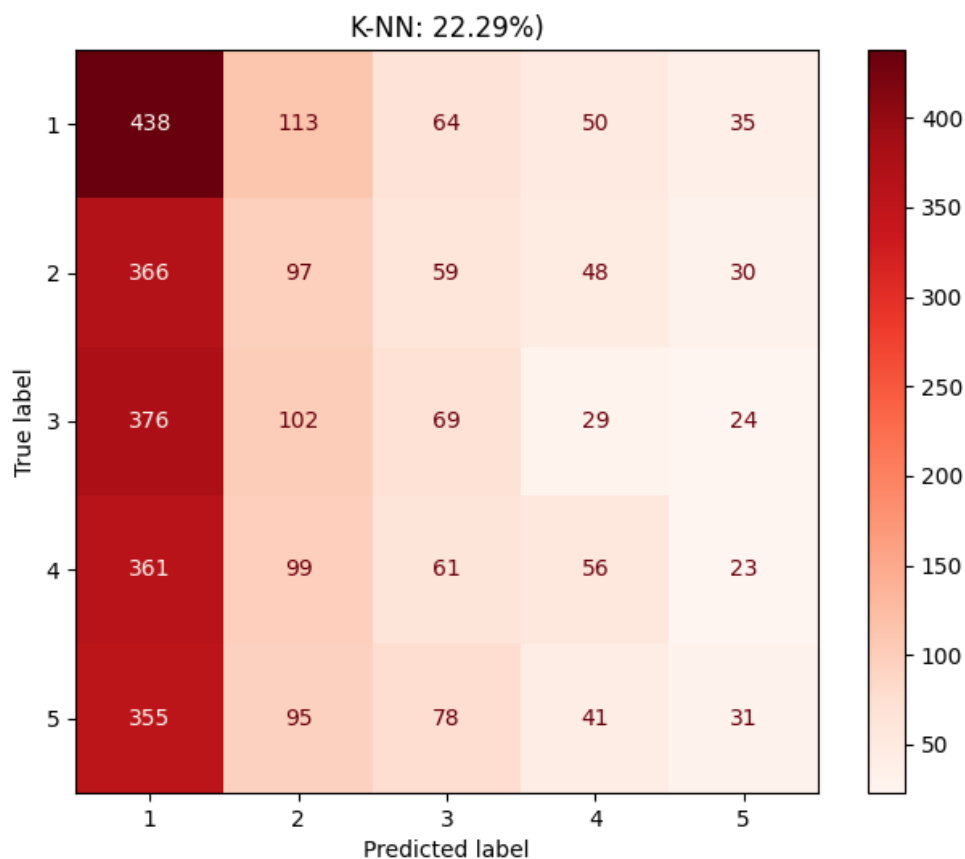
model = knn(train_images, train_labels)

predictii = []
for i in range(len(test_images)):
    predictie = model.clasificare(test_images[i], k=5, metrica='l2')
    predictii.append(predictie)

test_df['label'] = predictii
test_df.to_csv('knn_sub.csv', index=False)

```

1.3 Confusion Matrix: Results



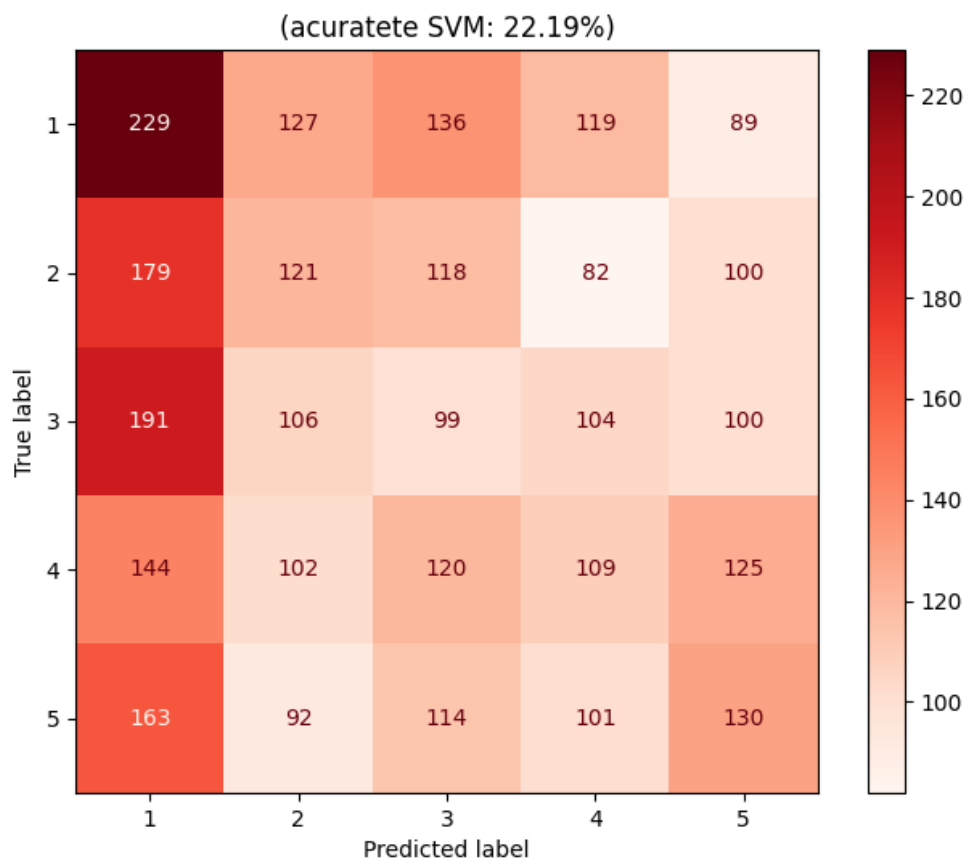
As we can see the K-NN algorithm predictions are also bad, probably due the noise in the picture the Euclidean Distance between data is completely random and np.argmax returns the first value it finds, explaining the bias for Class 1.

2 Second Attempt: SVM

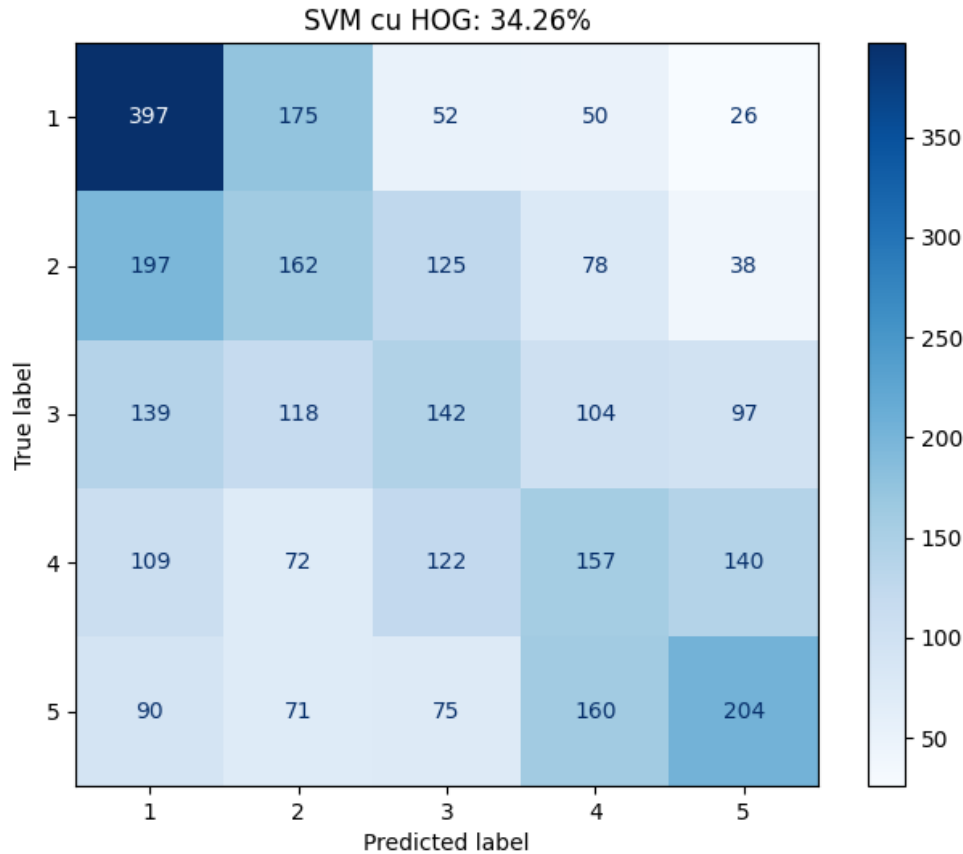
2.1 About the approach

My initial approach was training a SVM directly on flattened pixels. This model performed even worse than the K-NN because flattening the image destroys features like edges or shapes. Trying to fix this problem I searched the internet for some feature extraction SVM and came across this approach using [Histogram of Oriented Gradients](#). This approach improved performance but I was not really satisfied.

2.2 Confusion Matrix for SVM with no feature extraction: Results



2.3 Confusion Matrix for SVM with HOG: Results



2.4 Python Code

```
import cv2
import numpy as np
import pandas as pd
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

traincsvpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-o
trainimgpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-
testcsvpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-o
testimgpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-ob
hog = cv2.HOGDescriptor((64,128),(16,16),(8,8),(8,8),9)
    #img dimensions (64,128)
    #blocks size(16,16)
    #blocks shift (8,8)
    #nr of bins: 9

traindata = pd.read_csv(traincsvpath)
testdata_df = pd.read_csv(testcsvpath)
traindata_dict = traindata.to_dict('records')
testdata_dict = testdata_df.to_dict('records')
trainfeatures = []
trainlabels = []
```

```

for row in traindata_dict:
    imgpath = trainimgpath+"/"+str(row['id'])
    img= cv2.imread(imgpath,cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (64, 128))
    features =hog.compute(img).flatten()
    trainfeatures.append(features)
    trainlabels.append(row['label'])

trainfeatures = np.array(trainfeatures)
trainlabels = np.array(trainlabels)
testfeatures =[]

for row in testdata_dict:
    imgpath = testimgpath+"/"+str(row['id'])
    img = cv2.imread(imgpath, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (64, 128))
    features = hog.compute(img).flatten()
    testfeatures.append(features)

testfeatures = np.array(testfeatures)
xtrain, xval, ytrain, yval = train_test_split(trainfeatures, trainlabels, test_size=0.2, random_state=2)
scaler = StandardScaler()
xtrain = scaler.fit_transform(xtrain)
xval = scaler.transform(xval)
testfeatures = scaler.transform(testfeatures)

model = SVC(kernel='rbf', C=10.0, gamma='scale', random_state=26)
model.fit(xtrain, ytrain)
valpreds = model.predict(xval)
ac=accuracy_score(yval, valpreds)
print(f"acuratete pe validare: {ac*100:.2f}%")
predictii= model.predict(testfeatures)
testdata_df['label']=predictii
testdata_df[['id', 'label']].to_csv('svm_hog_sub.csv', index=False)

```

As we can see the accuracy is really bad, the model basically learned nothing and is making random precisions withing 5 classes. Due to the spatial features of the pictures, flattening the pictures into vectors that the SVM needs destroys every important feature of the data.

3 Last Attempt: CNN

3.1 Why I have choose this

After understanding the data properly and how the model needs to learn the features of classes by itself(classes are not defined by straight forward features like cat, dogs, cars classifications are) I made some research on what's the best model for this kind of problem. I came to the conclusion one of the best models is a Deep Learning Convolution Neural Network. I watched [this Youtube playlist \[1\]](#) for a better understanding of what a CNN is, what convolution layers are, how the data is best pre-processed and normalized and how the fully connected layer classifies the data correctly. For how PyTorch methods works I read this [PyTorch 60 minutes blitz](#) and helped myself with [full PyTorch documentation](#).

3.2 Data Preprocessing: CLAHE

For this type of data I've learned a Contrast Limited Adaptive Histogram Equalization (CLAHE) is better than the Z-score Normalization that fails to increase the contrast between the faint signals and the background. CLAHE divides the picture in 8x8 grids and equilize the contrast locally so the already hard to see signals are visible without making the already bright pixels brighter. This link was pretty useful for a broad understanding: [CLAHE Histogram Equalization - OpenCV](#)

3.3 Data Augmentation

Basically what I did, I applied a 50% chance of the image to be flipped vertically and/or horizontally before being fed to the model preventing the model to overfit and learn actual features not pixels.

3.4 Neural Network Architecture

Assuming one of the rules is we can't use pre-trained models, I've searched what kind of model is best for this problem and after reading [Introduction To Neural Networks](#) and [Introduction to Convolution Neural Network](#) I build a simple CNN with visible and hidden layers.

3.4.1 Visible Input First Layer

The first layer is a visible input layer with these dimensions:

1: 1 channel for the grayscale

128: height of the pictures

58: width of the pictures

[changed in later tuning]

3.4.2 Hidden Convolution Blocks

The first block slides a 3x3 window(kernel) **[changed in later tuning]** across the image to detect basic edges and shapes. This allows the network to detect spatial features and not pixels. This created 16 "new feature maps" increasing the channels for the next layers to 16.

At first I didn't use BatchNorm2D, forcing the numbers to have a mean of 0 and a variance of 1, and had this problem where all numbers became negative causing the gradients to die and the loss was stuck on 1.609 every epoch.

I used the non-linear ReLU activation function.

MaxPool2D slides a 2 x 2 grid over those feature maps and over those 4 pixels it chooses the maximum value. This shrinks the image to a 64 x 27 one forcing the network to learn bigger structures and creates spatial invariance. I tried Average Pooling too but the Max Pooling result was better, probably due to the AvgPool making the already noise-full pictures being more blurry.

The next two blocks basically are doing basically the same thing as the first one:

Block 2 increases channels from 16 to 32 and shrinks dimensions from 64 x 27 to 32 x 13.

Block 3 increases channels from 32 to 64 and shrinks dimensions from 32 x 13 to 16 x 6.

[changed in later tuning]

creating a 64 deep "feature map" of 16x16 size from the original image.

3.4.3 Hidden Global Average Pooling

Instead of feeding a massive array of 64 * 16 * 6 size to the fully connected layer we calculate the average of each element in the feature map reducing the size to [64, 1, 1]. With this technique we also prevent overfitting and translation invariance(the network doesn't care where the signal is but if it exists or not).

3.4.4 Visible Output Classification Layer

This is a fully connected linear layer that takes the 64 features generated by the previous 3 blocks and classifies the image using the 5 output neurons. The size changes from 64 to 5 after this.

3.5 Dropout

To prevent overfitting I added a Dropout layer with a probability of $p = 0.4$ before the fully connected layer. This means that during training, 40% of the neurons are randomly set to zero at each forward pass, forcing the network to not rely on any single feature.

3.6 Training and Optimization

For the optimizer I used the SGD [**changed in later tuning**] with a learning rate of 0.01 and momentum of 0.9. Important to mention in my past laboratory work I used this optimizer without the momentum parameter and for this problem that made my validation accuracy stuck between 22% and 27%, basically the neural network got stuck and didn't learn anything, it was choosing randomly from 5 options.

3.7 Label Smoothing Loss

[Label smoothing](#) is a technique used in machine learning particularly in neural networks to improve the generalization and robustness of the classification models. Adjusting the target labels during the training label smoothing helps prevent the model from becoming too confident in its predictions which can lead to overfitting and poor generalization.

3.8 Learning Rate Scheduler

During training, I noticed disproportionate validation loss because a static learning rate caused the optimizer to constantly overshoot the minimum. To fix this, I implemented a [Cosine Annealing Learning Rate Schedule](#). Using it to shrink the learning rate down to a very small step size near the end of training. I allowed the optimizer to settle into the absolute minimum without bouncing out, which completely stabilized my validation accuracy.

3.9 K-Fold Cross Validation

For training I first separated the data into 80% training and 20% validation. Of course that worked with a 0.59 Kaggle score, but I knew there is the K-Folds Cross Validation method which divides the data into 5 folds and we train 5 CNN. Model 1 trains on folds 2-5 and checks on 1, Model 2 Trains on 1, 3-5 and checks on 2 and so on. This ensures zero data waste and prevents unbalanced validation sets (ex. there is no Class 4 in the first 80% of the training data).

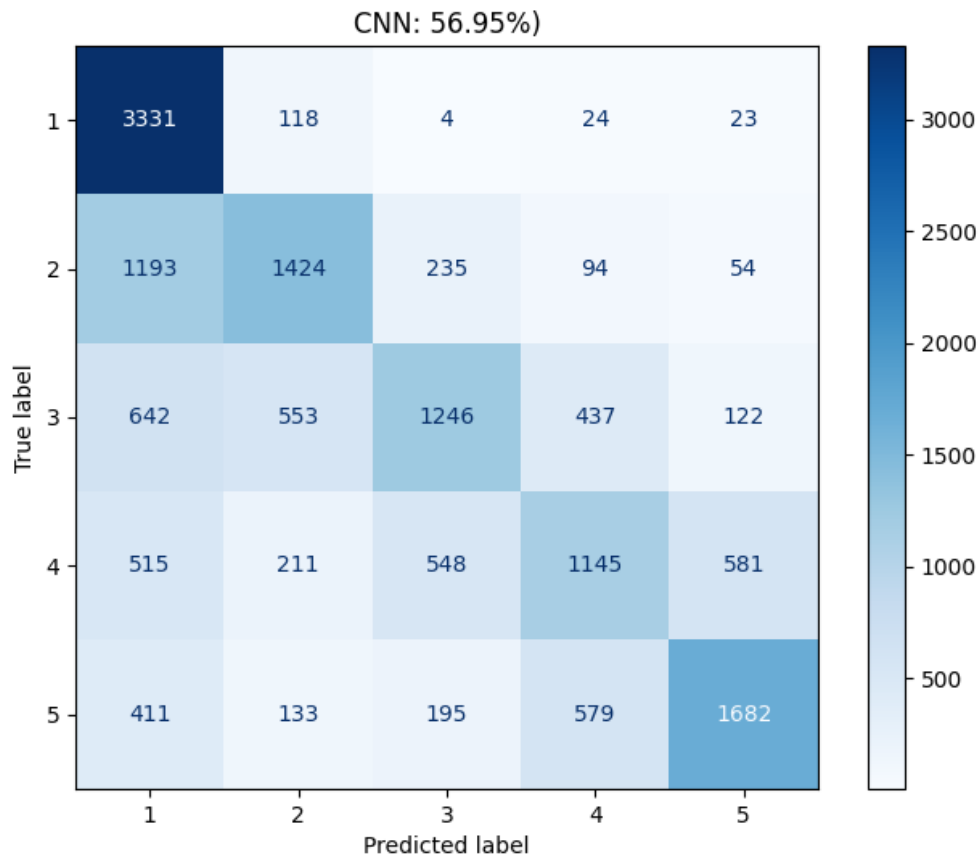
Beyond just validating with K-Fold, I also used the 5 trained models to ensemble a better solution. For each test image, all 5 models produce a probability distribution over the 5 classes. These distributions are summed across all folds into `foldspredtest` and the final prediction is the class with the highest total probability.

This boosted my Kaggle score to 0.65.

3.10 Early Stopping

Instead of training on a fixed number of epochs I've chosen to dynamically check in Validation Loss Improves in a span of 15 epochs. This prevents the model from memorizing the noise in the training set. I set a margin of 0.01 between the current validation loss and the best one so far so little change does not reset the counter. The model saves and restores the weights of the best epoch, ensuring the best model is always used for the testing data.

3.11 Confusion Matrix: Results



As we can see, this model runs so much better than K-NN and SVM. But I also notice these next interesting things:

1. The model has this bias of predicting the other classes for the Class 1. After running this script that shows the distribution of pictures between classes I found the Class 2, 3, 4, 5 have 3000 pictures but the first one has 500 more, that's probably where the bias comes from. With that in mind we can use weighted loss on the loss function so Class 1 gets lower weight since it's over-represented.

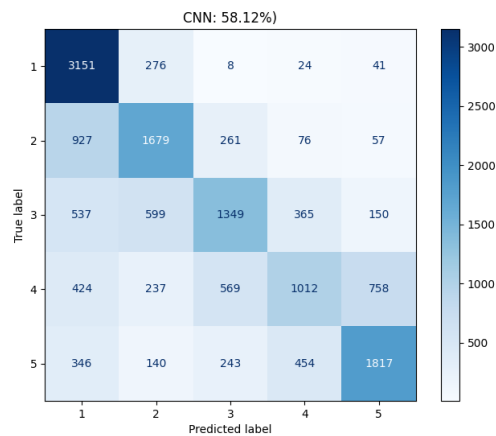
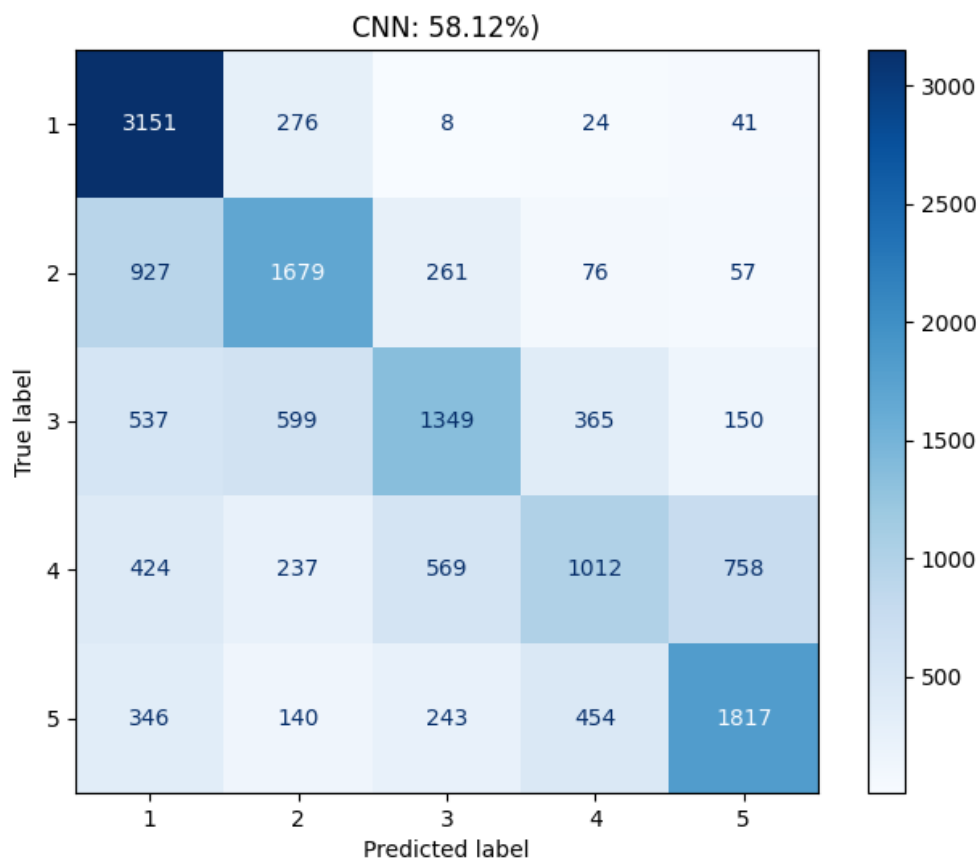


Figure 1: Enter Caption



2. There might be another problem with Class 1, it being a fit-all-class which I don't know how to solve yet ;(.

3. Errors are bigger within the Classes next to each other. Class 2 is mistaken for Class 1 and Class 3, Class 2 for 3 and 4 and Class 4 for 3 and 5. The model though never mistakes Class 1 for Class 5 and CrossEntropy ignores this. I'm going to keep this in mind for further reference.

3.12 Trying to optimize this model

I'm more or less satisfied with the 0.65 Kaggle result but I have 5 more days to go until the deadline so I'm searching on the internet some more tricks for optimizing your model and after visiting [this website](#) I came to the conclusion that there are some things I haven't tried and those are **Weight Decay** and **Other Optimizers**.

3.12.1 Trying Optimizers

I made this script that basically takes my code and changes the optimizer with the ones recommended on [this website](#). I'm going to try these: **SGD**, **Adam**, **AdamW**, **RMSprop**, **Adagrad** with 35 epochs and I'm going to pause the k-folds for now and go back to the 80% Training and 20% Validation Technique to save time.

The Results:

Optimizer	Validation Accuracy
SGD with Learning Rate of 0.01 and Momentum of 0.9	49%
Adam	56%
AdamW	55%
RMSprop	53%
Adagrad	51%

So for now, the ADM optimizer is the way to go.

3.12.2 Trying Learning Rates

Learning Rate	Accuracy
0.001	56%
0.0003	53%
0.0001	45%

3.12.3 Trying different Schedulers

Weight Decay Value	Accuracy
StepLR	56%
ReduceLROnPlateau	55%
CosineAnnealingLR	55%
OneCycleLR	51%

3.12.4 Trying Weight Decays

For this step I'm going to try some tuning I've found relevant on internet and even this one I've found on [this website](#) and never used before `optimizer = o.SGD(model.parameters(), lr=0.01, momentum=0.9, weight_decay = 1e-4 / epoch)` on my initial Early Stopping K-Folds CNN model.

Weight Decay Value	Accuracy
1e-4	53%
1e-3	54%
1e-2	48%
1e-4 / epoch	51%

3.13 Gradient Clipping

During training I used `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` after the backward pass and before the optimizer step. This caps the norm of the gradients at 1.0, preventing the **exploding gradients** problem where a single bad batch causes large weight updates that messes with training. I found this particularly useful in combination with the CosineAnnealingLR scheduler.

3.14 Test Time Augmentation

For each test image, instead of feeding it to the model once, I feed it **twice**: once as the original and once horizontally flipped. I then apply **softmax** on both outputs and average the two probability before taking the prediction.

```
p1 = torch.softmax(model(img), dim=1)
img_flip = torch.flip(img, dims=[3])
p2 = torch.softmax(model(img_flip), dim=1)
preds = (p1 + p2) / 2.0
```

3.15 Another Convolution Layer

The deadline for the competition was approaching so in a panic to get a better score I thought about adding more convolution layers. So for this step I added 2 more blocks hoping for a better score.

Block 4 increases channels from 64 to 128 and shrinks dimensions from 16 x 6 to 8 x 3.

Block 5 increases channels from 128 to 256 but **skips** the MaxPool2D step, leaving the spatial dimensions at 8 x 3.

3.16 Different Kernel Sizes

One last trick I wanted to try was changing the kernel size of the first convolution block to a vertical one, thinking I'll get a better score since the signals in the pictures were mostly on the Y axis or diagonal, but still pretty much vertical. I have chosen for the first block kernel's size to be 7 x 3.

3.17 Python Code

```
import random
import copy
import cv2
import numpy as np
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim

def normdistribpoze(device):
    picsperclass= np.array([3500, 3000,3000, 3000,3000], dtype=np.float32)
    distr = 1.0 /picsperclass
    distr /= distr.sum() * 5
    return torch.tensor(distr, dtype=torch.float32).to(device)

class myDataset(Dataset):
    def __init__(self, datadic, imgdir, train = True):
        self.pics= datadic
        self.imgdir = imgdir
        self.train = train

    def __len__(self):
        return len(self.pics)

    def __getitem__(self, item):
        curing = self.pics[item]
        path = self.imgdir + "/" + curing['id']
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```

img = clahe.apply(img)
img =img.astype(np.float32)/255.0

if self.train:
    if random.random()>0.5:
        img = copy.deepcopy(np.fliplr(img))
    # if random.random()>0.5:
    #     img = copy.deepcopy(np.flip(img, 0))

img = np.expand_dims(img, axis=0)

if 'label' in curing:
    return torch.tensor(img, dtype=torch.float32), torch.tensor(curing['label']-1, dtype=torch.
else:
    return torch.tensor(img, dtype=torch.float32), curing['id']

class myCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.s0 = nn.Conv2d(1, 16, kernel_size=(7,3), padding=(3,1))
        self.b0 = nn.BatchNorm2d(16)
        self.p0 = nn.MaxPool2d(2)

        self.s1 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.b1 = nn.BatchNorm2d(32)
        self.p1 = nn.MaxPool2d(2)

        self.s2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.b2 = nn.BatchNorm2d(64)
        self.p2 = nn.MaxPool2d(2)

        self.s3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.b3 = nn.BatchNorm2d(128)
        self.p3 = nn.MaxPool2d(2)

        self.s4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.b4 = nn.BatchNorm2d(256)

        self.relu = nn.ReLU()
        self.gap =nn.AdaptiveAvgPool2d(1)
        self.drop = nn.Dropout(p=0.4)
        self.fc = nn.Linear(256,5)

    def forward(self, x):

        x = self.s0(x)
        x = self.b0(x)
        x = self.relu(x)
        x = self.p0(x)

        x = self.s1(x)
        x = self.b1(x)
        x = self.relu(x)
        x = self.p1(x)

        x = self.s2(x)
        x = self.b2(x)
        x = self.relu(x)

```

```

        x = self.p2(x)

        x = self.s3(x)
        x = self.b3(x)
        x = self.relu(x)
        x = self.p3(x)

        x = self.s4(x)
        x = self.b4(x)
        x = self.relu(x)

        x = self.gap(x)
        x = x.view(x.size(0), -1)
        x = self.drop(x)
        x = self.fc(x)
        return x

traincsvpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-o
trainimgpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-
testcsvpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-o
testimgpath = "/Users/gabrielrachieru/PycharmProjects/IA-FMI-2026/SignalObjectDetection-Kaggle/signal-ob

data = pd.read_csv(traincsvpath)
data= data.sample(frac=1, random_state=26).reset_index(drop=True)
datadic = data.to_dict('records')

testdata = pd.read_csv(testcsvpath)
testdic = testdata.to_dict('records')
testDataset = myDataset(testdic, testimgpath, False)
testLoader = DataLoader(testDataset, batch_size=128, shuffle=False)

ntest=len(testdata)
foldspredstest = np.zeros((ntest, 5))
nrfolds=5
ndata=len(data)
foldsize = ndata //nrfolds
device ="mps"
epch =80

valpred = []
vallabels = []

classesweights = normdistribpoze(data, device)

for i in range(nrfolds):
    istart = i * foldsize
    if i < nrfolds - 1:
        ifinal = (i+1) * foldsize
    else:
        ifinal = ndata

    print(f"-----fold nr {i}-----")
    validationdata = datadic[istart:ifinal]
    traindata = datadic[:istart] + datadic[ifinal:]

    trainDataset = myDataset(traindata, trainimgpath, True)
    validationDataset = myDataset(validationdata, trainimgpath, False)

```

```
trainLoader = DataLoader(trainDataset, batch_size=128, shuffle=True)
validationLoader = DataLoader(validationDataset, batch_size=128, shuffle=False)

model = myCNN().to(device)

criterion = nn.CrossEntropyLoss(weight=classesweights, label_smoothing=0.05)
optimizer = o.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epch, eta_min=1e-5)

bestacc = 0.0
bestloss = float('inf')

nrepchswithnochange = 0

for e in range(epch):
    model.train()
    loss = 0.0
    for img, label in trainLoader:
        img = img.to(device)
        label = label.to(device)

        optimizer.zero_grad()
        preds = model(img)
        curloss = criterion(preds, label)
        curloss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        loss+= curloss.item()

    model.eval()
    validationloss = 0.0
    rightguess = 0
    total = 0

    with torch.no_grad():
        for img, label in validationLoader:
            img = img.to(device)
            label = label.to(device)

            preds = model(img)
            curloss = criterion(preds, label)
            validationloss+=curloss.item()

            dummy, guess = torch.max(preds, 1)
            total +=label.size(0)

            for j in range(len(guess)):
                if guess[j] == label[j]:
                    rightguess += 1

    avgvalidationloss = validationloss / len(validationLoader)
    acc = (rightguess / total) *100

    if bestloss - avgvalidationloss > 0.01:
        bestloss = avgvalidationloss
        nrepchswithnochange = 0
        torch.save(model.state_dict(), f"model_fold_{i}.pt")
```



```

else:
    if nrepchswithnochange == 15:
        break
    else:
        nrepchswithnochange +=1

if acc-bestacc > 0.01:
    bestacc =acc
    torch.save(model.state_dict(), f"model_fold_{i}.pt")

scheduler.step()
print(f"epoca{e+1} are acuratetea {acc:.2f}%")

model.load_state_dict(torch.load(f"model_fold_{i}.pt", weights_only=True))
model.eval()

with torch.no_grad():
    for img, label in validationLoader:
        img = img.to(device)
        preds = model(img)
        dummy, guess =torch.max(preds, 1)
        valpred.extend(guess.cpu().tolist())
        vallabels.extend(label.cpu().tolist())

predfold = []
with torch.no_grad():
    for img, id in testLoader:
        img = img.to(device)
        p1 = torch.softmax(model(img), dim=1)
        img_flip = torch.flip(img, dims=[3])
        p2 = torch.softmax(model(img_flip), dim=1)
        preds = (p1 + p2) / 2.0
        predfold.extend(preds.cpu().tolist())

foldspredstest += np.array(predfold)

finalprediction = np.argmax(foldspredstest, axis=1)+ 1
testdata['label'] = finalprediction
testdata[['id','label']].to_csv('submisie.csv', index=False)

```

4 Final Results

